

Exp 3:implement Merkle root from the transactions

Name : Abhishek Pandey

SAP : 60019220110

Roll No : B073

```
def verify_merkle_proof(tx, proof, root):
    """
    Verifies that a transaction is in the Merkle tree using the proof and root.
    """
    current_hash = hash_data(tx)
    for sibling in proof:
        if current_hash < sibling:
            current_hash = hash_data(current_hash + sibling)
        else:
            current_hash = hash_data(sibling + current_hash)
    return current_hash == root

# --- Block and Blockchain ---

class Block:
    def __init__(self, tx_list, prev_hash):
        self.ts = time.time()
        self.tx_list = tx_list
        self.prev_hash = prev_hash
        self.merkle_root = merkle_root(tx_list)
        self.hash = self.calc_hash()

    def calc_hash(self):
        s = str(self.ts) + str(self.merkle_root) + str(self.prev_hash)
        return hash_data(s)

class Blockchain:
    def __init__(self):
        self.chain = []
        self.genesis()

    def genesis(self):
        self.chain.append(Block(["Genesis Block"], "0"))

    def add(self, tx_list):
        prev_hash = self.chain[-1].hash
        self.chain.append(Block(tx_list, prev_hash))

    def show(self):
        for b in self.chain:
            print(f"Time          : {b.ts}")
            print(f"Transactions: {b.tx_list}")
            print(f"Merkle Root  : {b.merkle_root}")
            print(f"Prev Hash    : {b.prev_hash}")
            print(f"Block Hash   : {b.hash}")
            print(".....")

    def verify_transaction(self, tx):
        for block in self.chain:
            if tx in block.tx_list:
                proof = get_merkle_proof(block.tx_list, tx)
                is_valid = verify_merkle_proof(tx, proof, block.merkle_root)
                return is_valid
        return False
```

```
# --- Example Usage ---

bc = Blockchain()
bc.add(["A pays B 10", "C pays D 20", "E pays F 30"])
bc.add(["X pays Y 100", "Y pays Z 50"])
bc.show()

# Test transaction verification
tx = "C pays D 20"
print(f"\nIs '{tx}' a valid transaction? {bc.verify_transaction(tx)}")

tx2 = "Fake pays Scam 999"
print(f"Is '{tx2}' a valid transaction? {bc.verify_transaction(tx2)}")
```

Output :

```
PS C:\Users\djsce.student\Desktop\ibct exp> python Exp3Merkleroot.py
Time      : 1755843844.1767936
Transactions: ['Genesis Block']
Merkle Root : 89eb0ac031a63d2421cd05a2fbe41f3ea35f5c3712ca839cbf6b85c4ee07b7a3
Prev Hash   : 0
Block Hash  : 8ccdb5e850dbc8cdc2adfc57fddb2c1152845772d918d1bdc448fd46db7982d4
.....
Time      : 1755843844.1768255
Transactions: ['A pays B 10', 'C pays D 20', 'E pays F 30']
Merkle Root : 3f137dd96f80bf2d7f4d9b5500a11ec689e9b8a26f647197cd2362bc3585e38b
Prev Hash   : 8ccdb5e850dbc8cdc2adfc57fddb2c1152845772d918d1bdc448fd46db7982d4
Block Hash  : 25ed19f48b60f3364cf8ed2bf3b9c0143d0d3d6bcc862b8aaa9a33a9cdedb0c9
.....
Time      : 1755843844.1768544
Transactions: ['X pays Y 100', 'Y pays Z 50']
Merkle Root : 3b8193ea958bde22d80014d86908975724174b2b19fda0c7da42f3bb25b7f578
Prev Hash   : 25ed19f48b60f3364cf8ed2bf3b9c0143d0d3d6bcc862b8aaa9a33a9cdedb0c9
Block Hash  : ba15122bd8c2aed376613a0fb1dce5ad90165621df12fd1b92c694fcaa865aad
.....

Is 'C pays D 20' a valid transaction? False
Is 'Fake pays Scam 999' a valid transaction? False
PS C:\Users\djsce.student\Desktop\ibct exp> █
```